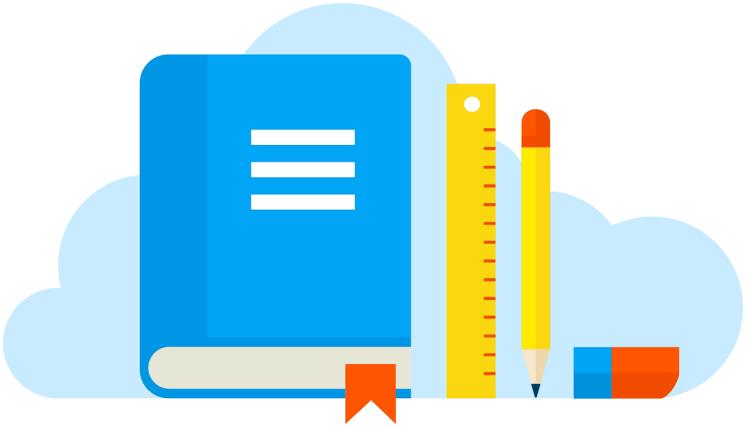


📘 NOTE 3 — REACT.JS INTRODUCTION

Date: 09/04/2026

REACT.JS INTRODUCTION

Web Development Notes

Subject: Web Development
Topic: React.js Introduction
Level: BCA Semester 5

---

1. What is React.js?

React.js is an open-source JavaScript library used to build user interfaces, especially for web applications.

React was developed by **Facebook (Meta)** and is widely used for creating modern, interactive, and reusable user interfaces.

React mainly focuses on the **frontend** part of an application.

Example

In StudySphere, React can be used to create:

- Student Dashboard
- Notes Page
- Assignment Page
- Profile Page
- Navigation Bar
- Admin Dashboard

---

2. Features of React.js

Important features of React include:

1. Component-Based

React applications are divided into small reusable components.

For example:

- Navbar
- ↓
- Dashboard
- ↓
- Notes
- ↓

Assignment

Each part can be developed as a separate component.

2. Reusable Components

A component can be created once and reused in different pages.

3. Virtual DOM

React uses a **Virtual DOM** to efficiently update the user interface when data changes.

4. JSX

React uses JSX, which allows developers to write HTML-like syntax inside JavaScript.

5. Fast Development

Reusable components and modern development tools make it easier to build applications.

---

3. What is a Component?

A **component** is a reusable building block of a React application.

For example, a website can have:

- App
- ├── Navbar
- ├── Home
- ├── Notes
- ├── Assignments

└── Footer

Each part can be implemented as a React component.

Simple Component

- function Welcome() {
- return <h1>Welcome to StudySphere</h1>;

```
}

export default Welcome;
```

This component displays a welcome message.

---

4. JSX

JSX stands for JavaScript XML.

JSX allows developers to write HTML-like code inside JavaScript.

Example:

```
function Home() {
  return (
    <div>
      <h1>StudySphere</h1>
      <p>Learn. Share. Succeed.</p>
    </div>
  );
}
```

JSX makes React code easier to understand because the UI structure can be written directly inside the component.

---

5. Creating a React Project

A React project can be created using **Vite**.

Basic command:

```
npm create vite@latest studysphere
```

Then install dependencies:

- cd studysphere

npm install

Start the development server:

npm run dev

The application can then be opened in the browser using the local address provided by Vite.

---

6. React Project Structure

A basic React project may contain:

- studysphere/
- │
- ├── public/
- ├── src/
- │ ├── components/
- │ ├── pages/
- │ ├── App.jsx
- │ ├── main.jsx
- │ └── index.css
- │
- ├── package.json

└── vite.config.js

Important folders

components/
Contains reusable components such as Navbar and Footer.

pages/
Contains application pages such as Home, Notes, and Assignments.

App.jsx
Main application component.

main.jsx
Entry point of the React application.

package.json
Contains project information and dependencies.

---

7. Props

Props are used to pass data from one component to another.

Example:

```
function Student({ name }) {
    return <h2>Hello, {name}</h2>;
}

function App() {
    return <Student name="Aarav" />;
}
```

Here, `name` is passed from the `App` component to the `Student` component.

Props help make components **reusable and dynamic**.

---

8. State

State is data that can change during the execution of a React application.

React provides the `useState()` Hook for managing state.

Example:

```
import { useState } from "react";

function Counter() {
  const [count, setCount] = useState(0);

  return (
    <button onClick={() => setCount(count + 1)}>
      Count: {count}
    </button>
  );
}
```

Whenever the button is clicked, the count increases.

---

9. React Hooks

Hooks are special functions that allow functional components to use React features.

Some commonly used Hooks are:

- `useState()` – manages state
- `useEffect()` – performs side effects
- `useContext()` – accesses shared context
- `useRef()` – stores references or values

Example:

```
useEffect(() => {
  console.log("Page loaded");
}, []);
```

---

10. Event Handling

React can handle user events such as:

- Click
- Submit
- Change
- Mouse events
- Keyboard events

Example:

```
function Button() {
    const handleClick = () => {
        alert("Welcome to StudySphere");
    };

    return (
        <button onClick={handleClick}>
            Click Me
        </button>
    );

}
```

The `onClick` event runs when the user clicks the button.

---

11. Conditional Rendering

Conditional rendering means displaying different content depending on a condition.

Example:

```
function LoginStatus({ loggedIn }) {
    if (loggedIn) {
        return <h2>Welcome Student</h2>;
    }

    return <h2>Please Login</h2>;
}
```

This can be useful in StudySphere for displaying different content to **logged-in and logged-out users**.

---

12. Lists in React

React can display multiple items using JavaScript's `map()` method.

Example:

```
const subjects = [
    "Web Development",
    "DBMS",
    "Computer Networks",
    "Operating Systems"
];

function Subjects() {
    return (
        <ul>
            {subjects.map((subject) => (
                <li key={subject}>{subject}</li>
            ))}
        </ul>
    );
}
```

This is useful for displaying lists of:

- Notes
- Assignments
- Students
- Subjects
- Submissions

---

13. React Router

React Router is used to create navigation between different pages in a React application.

For example:

- / → Home
- /login → Login
- /register → Register
- /student/notes → Notes
- /student/assignments → Assignments

/admin/notes → Admin Notes

It allows a React application to display different components based on the URL.

---

14. React in StudySphere

React is used in the MERN version of StudySphere to create the frontend interface.

For example:

- React
- │
- ├── Navbar

- ├── Home
- ├── Student Dashboard
- ├── Notes
- ├── Assignments
- ├── Profile

└── Admin Dashboard

React communicates with the backend through APIs.

The overall flow is:

- Student
- ↓
- React Frontend
- ↓
- Node.js / Express Backend
- ↓

MongoDB

For example, when a student opens the Notes page, React can request notes from the backend API and display them on the screen.

---

15. Advantages of React.js

React provides several advantages:

- Reusable components
- Fast UI updates
- Easy development of interactive interfaces
- Large developer community
- Component-based architecture
- Easy integration with APIs
- Suitable for single-page applications

16. Conclusion

React.js is an important JavaScript library for developing modern user interfaces.

Its **components, JSX, props, state, Hooks, event handling, and routing** make it suitable for developing applications such as StudySphere.

In the MERN stack:

React → Frontend
Node.js + Express → Backend
MongoDB → Database